Koneksi ke Database

Pemrograman Web • 11 slides • Generated on 11 October 2026, 22:42

Koneksi ke Database

- **Materi:**
  - `database/sql` dan driver MySQL
  - DSN, konfigurasi dari environment variable
  - `*sql.DB` sebagai connection pool
  - Cek koneksi saat startup dengan `PingContext`
  - Query SELECT pertama: `Query`, `Next`, `Scan`
  - Menangani nilai `NULL`

Auto Reload Server

Slide 2

```
go install github.com/bokwoon95/wgo@latest

wgo run main.go                 # alternatif "go run main.go" agar auto reload saat ada file change
wgo run -file .html main.go     # ikut memantau perubahan file .html
```

Jika wgo tidak dikenali

- Windows

```
- Tekan tombol Windows, ketik "environment variables", lalu pilih Edit the system environment variables.
- Klik Environment Variables...
- Di bagian User variables, pilih Path, lalu klik Edit, kemudian New.
- Isi %USERPROFILE%\go\bin, lalu klik OK sampai semua jendela tertutup.
- Tutup dan buka ulang terminal
```

- Mac

```
echo 'export PATH="$PATH:$(go env GOPATH)/bin"' >> ~/.zshrc
source ~/.zshrc
```

`database/sql` dan Driver

Slide 3

- `database/sql` = package standar Go untuk semua RDBMS
- Driver = implementasi untuk database tertentu

```
go get github.com/go-sql-driver/mysql
```

```
import (
    "database/sql"

    _ "github.com/go-sql-driver/mysql"
)
```

- `_` = import hanya untuk efek samping: driver mendaftarkan dirinya sebagai `"mysql"`
- Kode kita tetap memakai API `database/sql`, driver bisa diganti

DSN: Alamat Database

Slide 4

- DSN (Data Source Name) untuk MySQL:

```
user:password@tcp(host:port)/nama_db?param=nilai
```

```
app:rahasia@tcp(127.0.0.1:3306)/toko?parseTime=true&charset=utf8mb4
```

- Parameter yang sebaiknya selalu ada:
  - `parseTime=true`: kolom waktu jadi `time.Time`, bukan `[]byte`
  - `charset=utf8mb4`: aman untuk emoji dan huruf non-latin

File .env (Development)

Slide 5

File `.env` di root project (**jangan di-commit**):

```
DB_DSN=app:rahasia@tcp(127.0.0.1:3306)/toko?parseTime=true&charset=utf8mb4
```

```
install dulu:
go get github.com/joho/godotenv
```

```
import "github.com/joho/godotenv"

func main() {
    _ = godotenv.Load() // baca .env jika ada

    dsn := os.Getenv("DB_DSN")
    if dsn == "" {
        log.Fatal("DB_DSN belum di-set (cek file .env)")
    }
}
```

sql.Open dan Connection Pool

Slide 6

```
db, err := sql.Open("mysql", dsn) // db bertipe *sql.DB
if err != nil {
    log.Fatal(err)
}
defer db.Close()
```

- `sql.Open` mengembalikan `*sql.DB`, "pintu" aplikasi ke database
- `sql.Open` **belum** menghubungi server, hanya menyiapkan objek
- `*sql.DB` bukan satu koneksi, tapi **pool** (kumpulan koneksi):
  - Query butuh koneksi akan ambil dari pool
  - Jika sudah selesai, koneksi dikembalikan, dipakai query berikutnya
  - Pool membuat koneksi bisa dipakai ulang (hemat)

Aturan Pakai Pool

Slide 7

- Buat **satu** `*sql.DB` di `main()` untuk seluruh aplikasi
- Aman dipakai banyak request sekaligus
- **Jangan** `sql.Open` di dalam handler (koneksi menumpuk karena tiap request buat pool baru)

Batasi ukuran pool:

```
db.SetMaxOpenConns(25)                 // maksimal koneksi terbuka
db.SetMaxIdleConns(25)                 // koneksi "nganggur" yang disimpan
db.SetConnMaxLifetime(5 * time.Minute) // ganti koneksi yang sudah lama
```

- Default `MaxOpenConns` = tanpa batas → traffic tinggi bisa menghabiskan jatah koneksi MySQL (*too many connections*)
- `ConnMaxLifetime` mencegah error koneksi yg basi setelah MySQL memutus koneksi lama

Cek Koneksi Saat Startup

Slide 8

- Karena `sql.Open` tidak konek, error baru muncul saat query pertama
- Lebih baik gagal **saat server nyala** daripada saat user pertama datang

```
ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
defer cancel()

if err := db.PingContext(ctx); err != nil {
    log.Fatalf("database tidak bisa dihubungi: %v", err)
}
log.Println("database terhubung")
```

- `context.WithTimeout`: batas waktu menunggu
  - Tanpa ini, server bisa menggantung lama kalau host DB salah
- Tidak perlu `Ping` di tiap request (tempatkan PING ini di `main()` saja)

Tabel Produk

Slide 9

```
CREATE TABLE produk (
    id         BIGINT AUTO_INCREMENT PRIMARY KEY,
    nama       VARCHAR(100) NOT NULL,
    harga      BIGINT NOT NULL,
    stok       INT NOT NULL DEFAULT 0,
    keterangan TEXT NULL,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
mysql -u app -p toko < db/schema.sql
```

```
INSERT INTO produk (nama, harga, stok, keterangan) VALUES
    ('Kopi Arabika Gayo 250g', 85000, 40, 'Biji kopi sangrai medium dari Aceh'),
    ('Teh Melati Tubruk', 12500, 120, NULL),
    ('Gula Aren Cair 500ml', 32000, 0, 'Stok habis, restock minggu depan'),
    ('Keripik Tempe Pedas', 15000, 75, NULL),
    ('Madu Hutan Sumbawa', 98000, 18, 'Madu murni, cocok untuk oleh-oleh');
```

Query Banyak Baris

Slide 10

```
type Produk struct {
    ID    int64  // BIGINT
    Name  string // VARCHAR
    Price int64  // BIGINT
    Stok  int    // INT
}

func ambilSemuaProduk(ctx context.Context, db *sql.DB) ([]Produk, error) {
    rows, err := db.QueryContext(ctx,
        "SELECT id, nama, harga, stok FROM produk ORDER BY id")
    if err != nil {
        return nil, err
    }
    defer rows.Close()

    var hasil []Produk
    for rows.Next() {
        var p Produk
        if err := rows.Scan(&p.ID, &p.Name, &p.Price, &p.Stok); err != nil {
            return nil, err
        }
        hasil = append(hasil, p)
    }
```

```
    return hasil, rows.Err()
}
```

- `defer rows.Close()` **wajib**. Kalau tidak ditulis, koneksi tidak kembali ke pool
- `Scan` butuh **pointer** (`&p.ID`), urutannya sama dengan urutan kolom di `SELECT`
- `rows.Err()` menangkap error yang terjadi di tengah iterasi
- Tulis kolom secara eksplisit, hindari `SELECT *`
    - Kalau kolom tabel bertambah, `Scan` tidak ikut rusak

Dipanggil dari handler, `ctx` berasal dari request:

```
func produkHandler(db *sql.DB) http.HandlerFunc {
 return func(w http.ResponseWriter, r *http.Request) {
  produk, err := ambilSemuaProduk(r.Context(), db)

  if err != nil {
   log.Fatal(err)
  }
  tplProduk.ExecuteTemplate(w, "layout", produk)
 }
}
```

Nilai `NULL`

Slide 11

- Kolom `keterangan TEXT NULL` bisa berisi `NULL`
- `Scan` ke `string` biasa → error:

```
converting NULL to string is unsupported
```

- Pilihan 1: tipe `sql.Null...`

```
var ket sql.NullString
rows.Scan(&p.ID, &ket)
if ket.Valid {
    p.Keterangan = ket.String
}
```

- Pilihan 2: ubah `NULL` jadi nilai default di SQL

```
SELECT id, COALESCE(keterangan, '') FROM produk
```

- Pilihan terbaik: rancang kolom `NOT NULL DEFAULT ''` kalau "kosong" sudah cukup
- `NULL` dipakai kalau "tidak diisi" memang beda arti dengan "kosong"